Operating Systems Assignment 2 – Task 2 (Q2)

CPU Scheduling – Cloud Video-Encoding Service (CO2, K5 – Evaluate, 8 marks). Environment: Ubuntu 24.04.5 LTS, gcc 13.3.0. Structure follows the assignment's expected flow: *Model* → *Design* → *Implement* → *Measure* → *Compare* → *Evaluate* → *Recommend*.

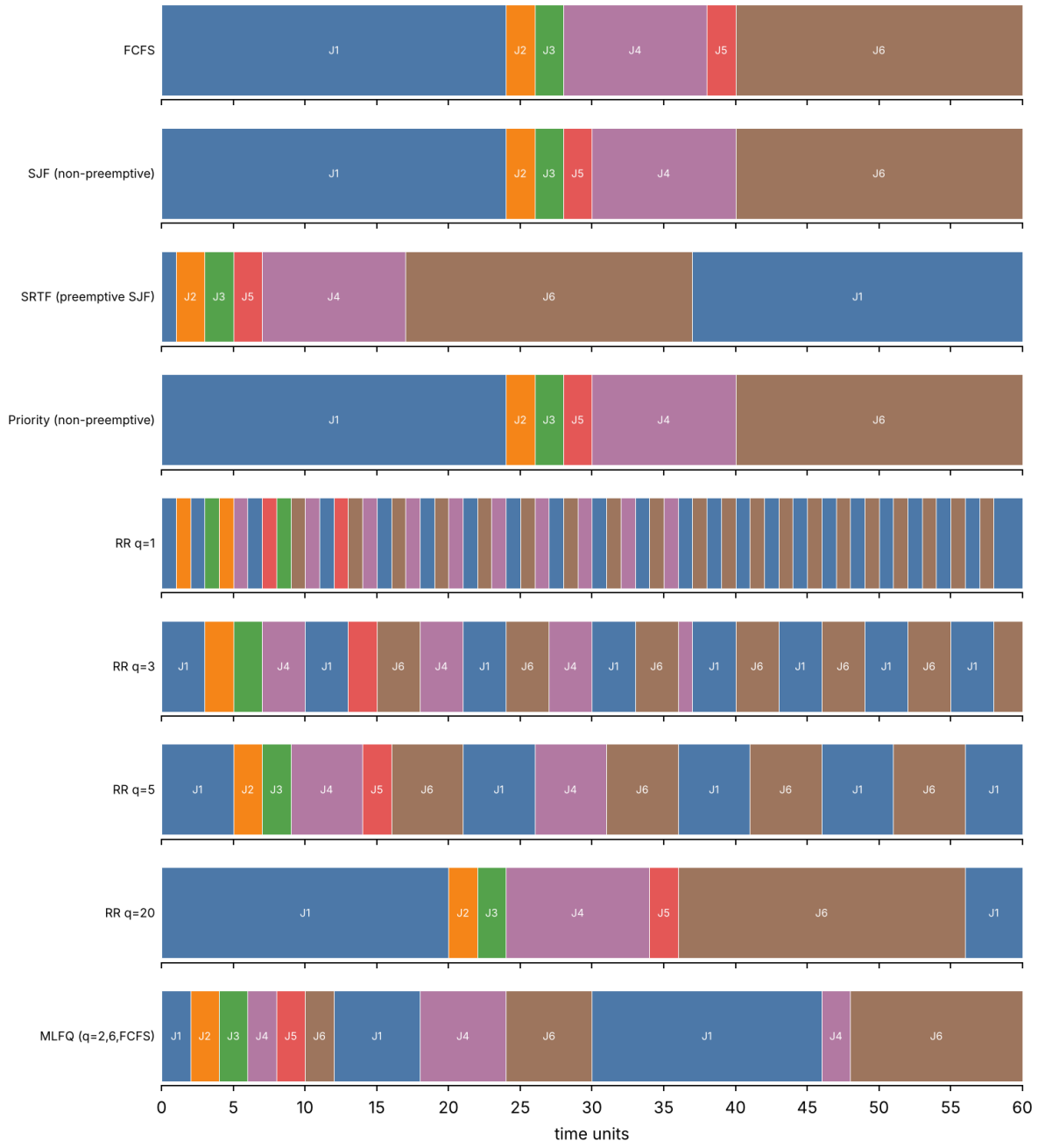
1. Model: the workload

The assignment does not supply a job table, so I model the service with six jobs (time unit = 1 s of encoding, scaled down; thumbnails 2 units, films 20+ units). Priority: lower number = higher priority. Total demand = 60 units, arrivals keep the CPU busy from time 0, so utilisation is 100% for every algorithm and the algorithms differ only in *who waits*.

Job	Type	Arrival	Burst	Priority
J1	Full film	0	24	3
J2	Thumbnail	1	2	1
J3	Thumbnail	2	2	1
J4	Medium clip	3	10	2
J5	Thumbnail	4	2	1
J6	Full film	5	20	3

Definitions: Turnaround (TAT) = completion – arrival; Waiting (WT) = TAT – burst; Response (RT) = first CPU time – arrival. A context switch is counted whenever the CPU passes to a different job. Priority scheduling is non-preemptive; Round Robin admits new arrivals before re-queueing the pre-empted job; MLFQ is defined in Section 6.

2. Design and results: Gantt charts, WT, TAT, RT



Gantt charts for all algorithms (computed by `sim.py`; RR also by the C program). Exact intervals and per-job values follow.

FCFS

`Gantt: |J1 0-24 |J2 24-26 |J3 26-28 |J4 28-38 |J5 38-40 |J6 40-60|`

Job	J1	J2	J3	J4	J5	J6	Average
WT	0	23	24	25	34	35	23.50
TAT	24	25	26	35	36	55	33.50
RT	0	23	24	25	34	35	23.50

Context switches: 5

SJF (non-preemptive)

`Gantt: |J1 0-24 |J2 24-26 |J3 26-28 |J5 28-30 |J4 30-40 |J6 40-60|`

Job	J1	J2	J3	J4	J5	J6	Average
WT	0	23	24	27	24	35	22.17

Job	J1	J2	J3	J4	J5	J6	Average
TAT	24	25	26	37	26	55	32.17
RT	0	23	24	27	24	35	22.17

Context switches: 5

SRTF (preemptive SJF)

```
Gantt: |J1 0-1 |J2 1-3 |J3 3-5 |J5 5-7 |J4 7-17 |J6 17-37 |J1 37-60|
```

Job	J1	J2	J3	J4	J5	J6	Average
WT	36	0	1	4	1	12	9.00
TAT	60	2	3	14	3	32	19.00
RT	0	0	1	4	1	12	3.00

Context switches: 6

Priority (non-preemptive)

```
Gantt: |J1 0-24 |J2 24-26 |J3 26-28 |J5 28-30 |J4 30-40 |J6 40-60|
```

Job	J1	J2	J3	J4	J5	J6	Average
WT	0	23	24	27	24	35	22.17
TAT	24	25	26	37	26	55	32.17
RT	0	23	24	27	24	35	22.17

Context switches: 5

RR q=1

```
Gantt: |J1 0-1 |J2 1-2 |J1 2-3 |J3 3-4 |J2 4-5 |J4 5-6 |J1 6-7 |J5 7-8 |J3 8-9 |J6 9-10 |J4 10-11 |J1 11-12 |J5 12-13
|J6 13-14 |J4 14-15 |J1 15-16 |J6 16-17 |J4 17-18 |J1 18-19 |J6 19-20 |J4 20-21 |J1 21-22 |J6 22-23 |J4 23-24 |J1 24-25
|J6 25-26 |J4 26-27 |J1 27-28 |J6 28-29 |J4 29-30 |J1 30-31 |J6 31-32 |J4 32-33 |J1 33-34 |J6 34-35 |J4 35-36 |J1 36-37
|J6 37-38 |J1 38-39 |J6 39-40 |J1 40-41 |J6 41-42 |J1 42-43 |J6 43-44 |J1 44-45 |J6 45-46 |J1 46-47 |J6 47-48 |J1 48-49
|J6 49-50 |J1 50-51 |J6 51-52 |J1 52-53 |J6 53-54 |J1 54-55 |J6 55-56 |J1 56-57 |J6 57-58 |J1 58-60|
```

Job	J1	J2	J3	J4	J5	J6	Average
WT	36	2	5	23	7	33	17.67
TAT	60	4	7	33	9	53	27.67
RT	0	0	1	2	3	4	1.67

Context switches: 58

RR q=3

```
Gantt: |J1 0-3 |J2 3-5 |J3 5-7 |J4 7-10 |J1 10-13 |J5 13-15 |J6 15-18 |J4 18-21 |J1 21-24 |J6 24-27 |J4 27-30 |J1 30-33
|J6 33-36 |J4 36-37 |J1 37-40 |J6 40-43 |J1 43-46 |J6 46-49 |J1 49-52 |J6 52-55 |J1 55-58 |J6 58-60|
```

Job	J1	J2	J3	J4	J5	J6	Average
WT	34	2	3	24	9	35	17.83
TAT	58	4	5	34	11	55	27.83
RT	0	2	3	4	9	10	4.67

Context switches: 21

RR q=5

```
Gantt: |J1 0-5 |J2 5-7 |J3 7-9 |J4 9-14 |J5 14-16 |J6 16-21 |J1 21-26 |J4 26-31 |J6 31-36 |J1 36-41 |J6 41-46 |J1 46-51
|J6 51-56 |J1 56-60|
```

Job	J1	J2	J3	J4	J5	J6	Average
WT	36	4	5	18	10	31	17.33
TAT	60	6	7	28	12	51	27.33
RT	0	4	5	6	10	11	6.00

Context switches: 13

RR q=20

Gantt: |J1 0-20 |J2 20-22 |J3 22-24 |J4 24-34 |J5 34-36 |J6 36-56 |J1 56-60|

Job	J1	J2	J3	J4	J5	J6	Average
WT	36	19	20	21	30	31	26.17
TAT	60	21	22	31	32	51	36.17
RT	0	19	20	21	30	31	20.17

Context switches: 6

MLFQ (q=2,6,FCFS)

Gantt: |J1 0-2 |J2 2-4 |J3 4-6 |J4 6-8 |J5 8-10 |J6 10-12 |J1 12-18 |J4 18-24 |J6 24-30 |J1 30-46 |J4 46-48 |J6 48-60|

Job	J1	J2	J3	J4	J5	J6	Average
WT	22	1	2	35	4	35	16.50
TAT	46	3	4	45	6	55	26.50
RT	0	1	2	3	4	5	2.50

Context switches: 11

3. Implement: Round Robin in C (Complete the Code, TODO 1)

TODO 1 was the dispatch step: take the head of the ready queue, run it for min(quantum, remaining), record first-run time, count context switches, add arrivals that came during the slice, then re-queue the job if unfinished. The rest of the program (job table, admission, statistics printing) is fixed. Because the assignment's skeleton was not included in the file I received, this is my own full program; if your faculty skeleton differs, paste the TODO block into it.

```
/* Task 2 - Round Robin scheduler (TODO 1 completed: the dispatch/quantum/requeue logic)
   Compile: gcc -std=c11 -Wall -Wextra -o rr rr.c      Run: ./rr &lt;quantum&gt; */
#include &lt;stdio.h&gt;
#include &lt;stdlib.h&gt;
#define N 6
typedef struct { const char *name; int at, bt, rem, first, ct; } Job;
int main(int argc, char *argv[]) {
    int q = (argc &gt; 1) ? atoi(argv[1]) : 3;
    if (q &lt;= 0) { fprintf(stderr, "quantum must be &gt; 0\n"); return 1; }
    Job j[N] = {{"J1-film",0,24,0,0,0},{"J2-thumb",1,2,0,0,0},{"J3-thumb",2,2,0,0,0},
                {"J4-medium",3,10,0,0,0},{"J5-thumb",4,2,0,0,0},{"J6-film",5,20,0,0,0}};
    int queue[1000], head = 0, tail = 0, t = 0, done = 0, next = 0, last = -1, cs = 0;
    for (int i = 0; i &lt; N; i++) { j[i].rem = j[i].bt; j[i].first = -1; }
    printf("Round Robin, quantum = %d\nGantt: ", q);
    while (done &lt; N) {
        while (next &lt; N &amp;&amp; j[next].at &lt;= t) queue[tail++] = next++;   /* admit arrivals */
        if (head == tail) { t++; continue; }                          /* CPU idle */
        /* TODO 1 (completed): dispatch head of queue, run for min(q, remaining) */
        int i = queue[head++];
        if (last != -1 &amp;&amp; last != i) cs++;                             /* context switch */
        last = i;
        if (j[i].first &lt; 0) j[i].first = t;                           /* response time */
        int run = (j[i].rem &lt; q) ? j[i].rem : q;
        printf("|%s %d-%d ", j[i].name, t, t + run);
        t += run; j[i].rem -= run;
        while (next &lt; N &amp;&amp; j[next].at &lt;= t) queue[tail++] = next++;   /* arrivals first ... */
        if (j[i].rem &gt; 0) queue[tail++] = i;                           /* ... then preempted job */
        else { j[i].ct = t; done++; }
    }
    printf("|\n\n%-10s %3s %3s %5s %5s %5s\n","Job","AT","BT","WT","TAT","RT");
    double w = 0, ta = 0, r = 0;
    for (int i = 0; i &lt; N; i++) {
        int tat = j[i].ct - j[i].at, wt = tat - j[i].bt, rt = j[i].first - j[i].at;
        printf("%-10s %3d %3d %5d %5d %5d\n", j[i].name, j[i].at, j[i].bt, wt, tat, rt);
        w += wt; ta += tat; r += rt;
    }
    printf("\nAvg WT=%.2f  Avg TAT=%.2f  Avg RT=%.2f  Context switches=%d\n", w/N, ta/N, r/N, cs);
    return 0;
}
```

Execution: quantum = 1 (full output)

```
Round Robin, quantum = 1
Gantt: |J1-film 0-1 |J2-thumb 1-2 |J1-film 2-3 |J3-thumb 3-4 |J2-thumb 4-5 |J4-medium 5-6 |J1-film 6-7 |J5-
thumb 7-8 |J3-thumb 8-9 |J6-film 9-10 |J4-medium 10-11 |J1-film 11-12 |J5-thumb 12-13 |J6-film 13-14 |J4-
medium 14-15 |J1-film 15-16 |J6-film 16-17 |J4-medium 17-18 |J1-film 18-19 |J6-film 19-20 |J4-medium 20-21
|J1-film 21-22 |J6-film 22-23 |J4-medium 23-24 |J1-film 24-25 |J6-film 25-26 |J4-medium 26-27 |J1-film 27-28
|J6-film 28-29 |J4-medium 29-30 |J1-film 30-31 |J6-film 31-32 |J4-medium 32-33 |J1-film 33-34 |J6-film 34-35
|J4-medium 35-36 |J1-film 36-37 |J6-film 37-38 |J1-film 38-39 |J6-film 39-40 |J1-film 40-41 |J6-film 41-42
|J1-film 42-43 |J6-film 43-44 |J1-film 44-45 |J6-film 45-46 |J1-film 46-47 |J6-film 47-48 |J1-film 48-49 |J6-
film 49-50 |J1-film 50-51 |J6-film 51-52 |J1-film 52-53 |J6-film 53-54 |J1-film 54-55 |J6-film 55-56 |J1-film
56-57 |J6-film 57-58 |J1-film 58-59 |J1-film 59-60 |

Job         AT  BT    WT   TAT    RT
J1-film      0  24    36    60     0
J2-thumb     1   2     2     4     0
J3-thumb     2   2     5     7     1
J4-medium    3  10    23    33     2
J5-thumb     4   2     7     9     3
J6-film      5  20    33    53     4

Avg WT=17.67  Avg TAT=27.67  Avg RT=1.67  Context switches=58
```

Execution: quantum = 20 (full output)

```
Round Robin, quantum = 20
Gantt: |J1-film 0-20 |J2-thumb 20-22 |J3-thumb 22-24 |J4-medium 24-34 |J5-thumb 34-36 |J6-film 36-56 |J1-film
56-60 |

Job         AT  BT    WT   TAT    RT
```

```
J1-film      0  24    36    60     0
J2-thumb     1   2    19    21    19
J3-thumb     2   2    20    22    20
J4-medium    3  10    21    31    21
J5-thumb     4   2    30    32    30
J6-film      5  20    31    51    31

Avg WT=26.17  Avg TAT=36.17  Avg RT=20.17  Context switches=6
```

The C results match the Python simulator exactly for q = 1, 3, 5, 20 (e.g. q=1: Avg WT 17.67, 58 switches; q=20: Avg WT 26.17, 6 switches).

4. Measure and compare

Algorithm	Avg WT	Avg TAT	Avg RT	Ctx sw	Thumbnail avg WT (J2,J3,J5)	Film avg WT (J1,J6)
FCFS	23.50	33.50	23.50	5	27.00	17.50
SJF (non-preemptive)	22.17	32.17	22.17	5	23.67	17.50
SRTF (preemptive SJF)	9.00	19.00	3.00	6	0.67	24.00
Priority (non-preemptive)	22.17	32.17	22.17	5	23.67	17.50
RR q=1	17.67	27.67	1.67	58	4.67	34.50
RR q=3	17.83	27.83	4.67	21	4.67	34.50
RR q=5	17.33	27.33	6.00	13	6.33	33.50
RR q=20	26.17	36.17	20.17	6	23.00	33.50
MLFQ (q=2,6,FCFS)	16.50	26.50	2.50	11	2.33	28.50

Reading the table. FCFS, non-preemptive SJF and priority all make thumbnails wait behind J1 (the convoy effect): thumbnail WT is 24–27 units, i.e. 12–13× their own 2-unit runtime — exactly the customers' complaint. SRTF gives the best averages (WT 9.00) because short jobs always run first, but it pushes the film J1 to the very end (WT 36). RR and MLFQ keep thumbnails at 2–6 units WT.

5. Evaluate

5.1 Effect of the Round Robin quantum (measured)

Quantum	Avg WT	Avg TAT	Avg RT	Context switches	Behaviour
1	17.67	27.67	1.67	58	Best response, but 58 switches; with a real switch cost the overhead would be large
3	17.83	27.83	4.67	21	Good balance; thumbnails (2 units) finish in their first slice
5	17.33	27.33	6.00	13	Lowest WT here, fewer switches, slower first response
20	26.17	36.17	20.17	6	Quantum ≥ most bursts → degenerates towards FCFS; thumbnails wait behind 20-unit slices

Small quantum → good response but many switches (overhead ∝ burst/q); large quantum → few switches but FCFS-like convoy behaviour. A common rule is to pick q slightly larger than the length of a typical short job (here q = 3–5, so thumbnails complete in one slice).

5.2 Starvation in SJF/priority, and ageing

SJF and priority always pick the best-ranked job, so a stream of short or high-priority jobs can postpone a long or low-priority job indefinitely. Demonstration (`starve.py`): a low-priority job L (burst 4, priority 5) arrives at time 0, while high-priority 2-unit jobs (priority 1) arrive every 2 units for 40 units, saturating the CPU, with pre-emptive priority scheduling.

Policy	L first runs at	L finishes at
Priority, no ageing	t = 40 (only when the stream ends)	44

Policy	L first runs at	L finishes at
Priority + ageing (priority number –1 per 5 units waited)	t = 20	24

Ageing raises a job's priority the longer it waits, so its waiting time is bounded (here after 20 units L reaches priority 1 and, being the oldest, wins ties). In the main job set, SJF would starve J1/J6 if short jobs kept arriving; non-preemptive SJF here avoided it only because the arrivals were finite.

5.3 How a multilevel feedback queue treats short and long jobs

New jobs enter the top queue with a short quantum. A job that finishes within it (thumbnail) leaves immediately with tiny wait. A job that uses its whole quantum is demoted to a queue with a longer quantum, and finally to a FCFS queue, so long jobs run in bigger slices (few switches) but only when higher queues are empty. Short/interactive jobs are therefore favoured without knowing burst lengths in advance (unlike SJF/SRTF).

6. Alternative concept: MLFQ instead of a single Round Robin queue

Design: Q0 quantum 2, Q1 quantum 6, Q2 FCFS. New jobs go to Q0; using the full quantum demotes a job one level; a higher queue pre-empts a lower one. Pseudocode:

```
on arrival(job):      job.level = 0; enqueue(Q[0], job)
schedule():           for L in 0..2: if Q[L] not empty: return dequeue(Q[L])   # at most 3 checks -> O(1)
on quantum_end(job):  if job.level < 2: job.level += 1;  enqueue(Q[job.level], job)
on arrival to higher level while lower job runs: pre-empt, keep job at head of its own queue
optional ageing:      if job waits > T: job.level -= 1
```

Metric (same job set)	Single RR q=3	Single RR q=5	MLFQ (2,6,FCFS)
Avg WT / TAT / RT	17.83 / 27.83 / 4.67	17.33 / 27.33 / 6.00	16.50 / 26.50 / 2.50
Thumbnails avg WT	4.67	6.33	2.33
Films (J1, J6) avg WT	34.50	33.50	28.50
Context switches	21	13	11

In this workload MLFQ gave short jobs the lowest wait (2.33), improved the films' wait too (28.5 vs ≈34) because slices grow and fewer switches occur, and had fewer switches than RR q=3. Caveat: this is one small job set; MLFQ's advantage is largest when many short jobs keep arriving.

Cost	SJF	Single RR	MLFQ
Decision time	O(n) scan of ready jobs (O(log n) with a heap)	O(1) (queue head)	O(1) per decision: ≤ k queue checks, k = 3 constant
Space	O(n)	O(n)	O(n) + O(k) queue headers
Needs burst length in advance?	Yes (must be predicted)	No	No (learns from behaviour)
Starvation risk	High for long jobs	None	Possible for long jobs in bottom queue → fix with ageing / periodic boost

Own measurement of per-decision selection cost (`scal.py`, Python in this sandbox; values illustrate growth, not C speed):

```
n=   1000  SJF scan      33.56 us   MLFQ/RR pick  0.135 us
n=  10000  SJF scan     306.81 us   MLFQ/RR pick  0.121 us
n= 100000  SJF scan    3633.33 us   MLFQ/RR pick  0.182 us
```

The SJF scan grows linearly (≈ 34 µs → 307 µs → 3.6 ms for ×10 more jobs) while the queue-based pick stays flat (≈ 0.12–0.18 µs), confirming O(n) vs O(1).

7. Recommend: scheduling policy for the video service

Recommendation: a multilevel feedback queue (Q0 q≈2, Q1 q≈6, Q2 FCFS) with ageing/periodic priority boost, on every server. Evidence from the calculated averages:

Policy	Avg WT	Thumbnail WT	Film WT	Verdict
FCFS	23.50	27.00	17.50	Convoy effect – thumbnails wait 27 units: rejected
SJF / Priority (non-preemptive)	22.17	23.67	17.50	Still blocked behind running film; starvation risk

Policy	Avg WT	Thumbnail WT	Film WT	Verdict
SRTF	9.00	0.67	24.00	Best averages, but needs burst prediction, many arrivals can starve films, high decision cost O(n)
RR q=3 / q=5	17.83 / 17.33	4.67 / 6.33	34.50 / 33.50	Fair and simple, but films pay heavily; q must be tuned
MLFQ	**16.50**	**2.33**	**28.50**	**Short jobs nearly immediate, no burst prediction, O(1) decisions, 11 switches**

It meets both goals: small jobs no longer wait behind films (WT 2.33 vs 27 under FCFS), and the CPU stays 100% busy because there is always work in some queue. Its weak point is starvation of long films when thumbnails keep arriving; the ageing/boost rule bounds that wait (Section 5.2). SRTF has the best raw average but is rejected because job lengths are not known exactly in a real service and long films would be starved.

Limitations

Results come from one 6-job scenario I defined (the assignment gave none); averages will change with a different job set. Context-switch *cost* is counted, not timed, and the decision-cost timing used Python. Run the programs yourself and commit them (`Task02/rr.c, sim.py, starve.py, scal.py`, outputs) as the assignment requires.